

## Lab 4: Interfacing Display Peripheral with Microcontroller

### Goals

1. Understanding of the fundamental concepts of Assembly Language
2. Able to write Simple Assembly program using ARMv7 ISA to interface with simple IO
3. Develop familiarity with a IO system of a Computer System Simulator

### Summary of Tasks:

1. Consult ARM Instruction Set Manual available at Canvas
2. Understand the following assembly example
  - a. The meaning of each of the identifiers in the program
  - b. How data segments and instruction segments work together
  - c. Understand the IO addressing and how to use it
3. Implement the following two program:
  - a. Display “Hello” and “World” separately in the seven segment display. When the passcode 0101 is pressed in the push button KEY[3..0]. The word will be replaced with the other one such as “Hello” will be replaced with “World”
  - b. Write another program that will continuously move “Hello World” within the six seven segment display. If a passcode 1010 is pressed, it will stop moving.
4. Do Something cool. The following are a few examples.
  - a. Display a moving “Welcome to Bangladesh”
  - b. Change passcode or integrate LEDs, switches to make something interesting

### Assembly program example:

```
/* *****  
* This program demonstrates the use of parallel ports in the DE1-SoC Computer  
* It performs the following:  
* 1. displays the SW switch values on the red lights LEDR  
* 2. displays a rotating pattern on the HEX displays  
* 3. if KEY[3..0] is pressed, uses the SW switches as the pattern  
***** */  
.org 1000 @program start at memory address 1000  
.text /* executable code follows */  
.global _start  
_start:  
    MOV R0, #31 // used to rotate a bit pattern: 31 positions to the  
// right is equivalent to 1 position to the left  
    LDR R1, =0xFF200000 // base address of LEDR lights  
    LDR R2, =0xFF200020 // base address of HEX3_HEX0 7-segs  
    LDR R3, =0xFF200040 // base address of SW switches  
    LDR R4, =0xFF200050 // base address of KEY pushbuttons  
    LDR R5, HEX_bits // load the initial pattern for the HEX displays
```

```
DO_DISPLAY: LDR R6, [R3] // load SW switches
            STR R6, [R1] // write to red LEDs
            LDR R7, [R4] // load pushbutton keys
            CMP R7, #0 // check if any key is pressed
            BEQ NO_BUTTON
            MOV R5, R6 // copy SW switch values onto HEX displays
WAIT:
            LDR R7, [R4] // load pushbuttons
            CMP R7, #0
            BNE WAIT // wait for KEY release
NO_BUTTON: STR R5, [R2] // store to HEX3 ... HEX0
            ROR R5, R0 // rotate the displayed pattern to the left
            LDR R6, =50000000 // delay counter
SUB_LOOP:  SUBS R6, R6, #1
            BNE SUB_LOOP
            B DO_DISPLAY
HEX_bits:
.word 0x0000000F // initial pattern for the HEX displays
.end
```

**Lab Demo**

**Lab 4: Interfacing Display Peripheral with Microcontroller**

Demo Part 3, Part 4 (optional) to lab instructor. And answer the following questions and submit a Canvas report

1. You can use ROR instruction to move between seven segment displays? How many ROR instruction you used and why?
2. Write a pseudo algorithm for ARM assembly that can simulate a traffic signal with LEDs. Consider the first LED as Green, second one as yellow and third one as red.
3. You can use NOP instruction and loop to simulate a delay function. Write a pseudo algorithm that with changing one parameter we can change the delay function. (Hint: you can use .equ pragma).